From Pixels to Sticks: What a Camera-Only FPV Acro Pilot Needs to Learn

Wee Kiat Chan

***Abstract*—FPV pilots fly fast drones by sight alone: a delayed, low-resolution video feed and four sticks, with no state estimate. Can a learned policy do the same? We train camera-only acro pilots in a fast JAX simulator with realistic latency, wind and sensor noise, by distilling a privileged teacher into a vision-based student. On targets 100–200 m away at up to 40 m/s, a controlled study over five seeds shows what such a pilot needs. Memory matters most, raising success from 44.8% to 66.5%. Training on the student's own flights adds 8–16 points, and RL fine-tuning a few more. The best student reaches 77.2% of targets (85.6% with 30 s), where a calibrated hand-designed pilot reaches 3.0%. Probing the network shows why memory helps: the frames already reveal attitude, but only memory recovers velocity. The whole pipeline trains on a single laptop CPU.**

I. Introduction

An FPV pilot flies in *acro* mode: the sticks command collective thrust and body *rates*, which the flight controller tracks, and everything else comes from an up-tilted, wide-angle video feed with 20–40 ms of latency. Autonomous agile flight has reached champion level [1], [2], but usually with an explicit state estimate. Agile flight directly from pixels has been demonstrated [3], and distilling privileged experts into sensorimotor students is the standard recipe [4]–[8]. What is still unclear is *which ingredients a camera-only pilot needs*. Disentangling them requires controlled, multi-seed experiments, which game-engine renderers and GPU-heavy training [9] make expensive.

We study this question in a transparent simulator that trains on one laptop CPU. Our contributions are:

1) A JAX-native pipeline for camera-only acro flight. It combines a rate-level quadrotor model with FPV non-idealities, a degraded ray-cast camera with active foveation and area-correct target rendering, and teacher–student learning in one compiled program.
2) A controlled long-range study over 5 seeds that crosses memory with three training stages (imitation, compute-matched further imitation, RL fine-tuning) and adds a hand-designed baseline. Memory is the robust ingredient; training on the student's own state distribution helps further; RL adds a small, consistent extra.
3) Linear probes with a memoryless control: what memory adds is a velocity estimate, not attitude and not the drone's parameters.
4) Short-range ablations validating the pipeline: DAgger vs. behaviour cloning, frame stacking, a perception-aware reward, domain randomisation.

Code: https://github.com/cweekiat/fpv-rl-jax

II. Related Work

Learning quadrotor control. RL controls quadrotors from state [10], [11], transfers across vehicles with dynamics randomisation [12], and can be trained in seconds [13]. Thrust-and-rates policies beat optimal control [2] and human champions [1]. We use the same action space because it is the human acro interface. **Vision-based agile flight.** Privileged experts have been distilled into students using feature tracks or depth [6], [7], perception-aware rewards keep goals in view [8], and pixel-based flight without state estimation has been shown on hardware [3]. We isolate the pixels-to-sticks problem under heavy degradation and attribute performance to individual ingredients. **Privileged learning.** Teacher–student methods [4], [5], [14], asymmetric actor–critic [15], DAgger [16] and domain randomisation [17], [18] are our building blocks; fine-tuning the student with RL addresses the imitation gap that remains when the student cannot observe what the teacher does. **Fast simulation.** Like massively parallel [19], [20] and JAX-native RL [21], [22], we compile everything, including the camera, into one XLA program.

III. Simulator and Tasks

Dynamics. A rigid body is integrated at 100 Hz; the policy acts at 50 Hz with $a = (a^T, a^\omega) \in [-1, 1]^4$. With mass-normalised thrust T,

$$\begin{aligned} T_{k+1} &= T_k + \tfrac{\Delta t}{\tau_m}(T^{cmd} - T_k), \quad T^{cmd} = \tfrac{1+a^T}{2}\lambda g\, b(t), \\ \dot{\boldsymbol{\omega}} &= \mathrm{clip}\big(\tfrac{\omega_{\max} a^\omega - \boldsymbol{\omega}}{\tau_r},\, \pm\alpha_{\max} h[1, 1, \kappa_y]\big) + \boldsymbol{\eta}_\omega, \\ \dot{\mathbf{v}} &= R(\mathbf{q})[0, 0, T\gamma(z)(1+\eta_T)]^\top - g\mathbf{e}_z \\ &\quad - c_1 \mathbf{v}_a - c_2 \|\mathbf{v}_a\| \mathbf{v}_a, \end{aligned}$$

with thrust-to-weight λ, battery factor $b(t)$, rate/motor time constants τ_r, τ_m, torque limit $\alpha_{\max}$, reduced yaw authority κ_y, saturation headroom h, airspeed $\mathbf{v}_a$ under wind and Ornstein–Uhlenbeck gusts, ground effect $\gamma(z)$, actuator noise, and 0–40 ms of control latency. Each episode samples 18 parameters (drone, wind, camera, video link; Table I) that no policy observes.

Camera. Per-pixel ray casting renders sky, a checker ground with mip-mapped texture and haze, and a dark spherical target, then applies per-episode exposure, pixel noise, 0–40 ms of video latency and random frame freezes. Rendering costs $\approx$15 μs per 32×24 image. Policies see the last three frames of every view.

Tasks. *Short range* (pipeline validation): targets 4–12 m away, success within 0.5 m, 5 s episodes, one 120° view. *Long range* (main study): targets 100–200 m away at 0–100 m altitude, success within 1 m, a drone reaching $\approx$40 m/s,

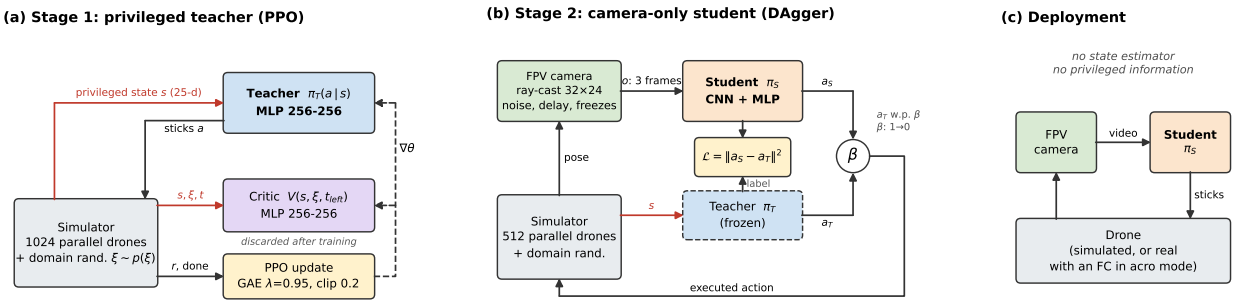


Fig. 1. Approach. (a) A privileged PPO teacher; its critic also sees the hidden randomisation ξ. (b) DAgger distils it into a student that sees only the FPV video and its own stick history. (c) The student is optionally trained further on its own observations (more DAgger or RL fine-tuning, Sec. IV) and is deployed with the camera alone.

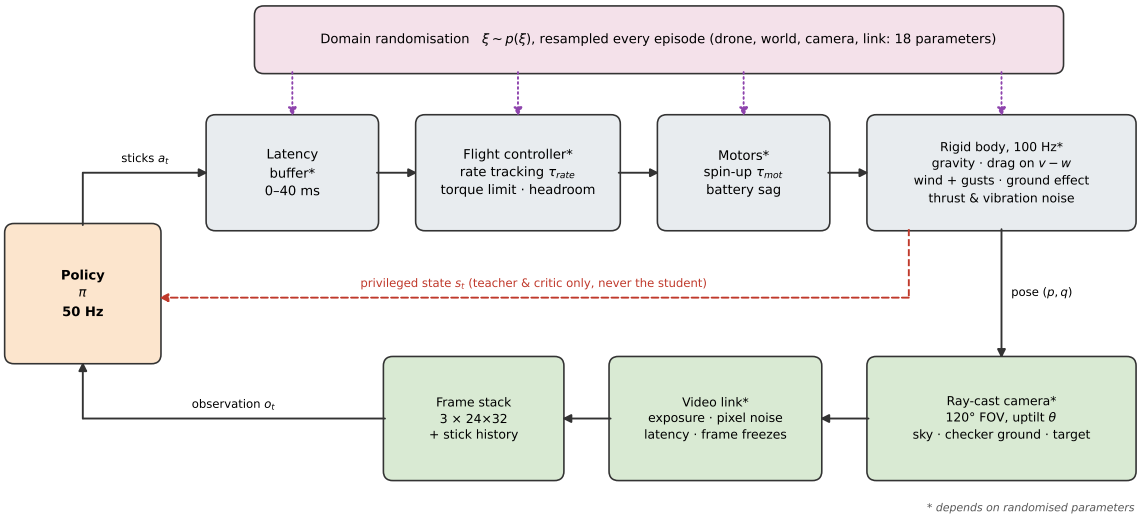


Fig. 2. Simulator loop. Sticks pass through a latency buffer, a torque-limited rate controller and lagged motors into a 100 Hz rigid-body model with aerodynamic and environmental effects; the pose is rendered by a ray-cast FPV camera and degraded by the video link. Blocks marked * depend on per-episode randomised parameters. The privileged state (dashed) reaches only the teacher and critic.

TABLE I
PER-EPISODE RANDOMISATION (LONG-RANGE VALUES IN BRACKETS)

thrust-to-weight	2.5–5 [4–7]	wind / gusts	0–3 / 0–1 m/s
τ_r / τ_m	20–70 / 15–50 ms	uptilt	20–40° [30–60°]
$\alpha_{\max}$, yaw auth.	80–250 rad/s^2, 0.3–0.6	video latency	0–40 ms
drag c_1 / c_2	0.1–0.5 / 0–0.05 [0.1–0.3 / 0.015–0.04]	exposure, black level	0.8–1.2, ±0.05
battery sag	0–3 %/s	pixel noise σ	0.01–0.05
control latency	0–40 ms	frame freezes	0–5 %

30 s training episodes with the battery floored at 70%, and evaluation on a standard 15 s benchmark (plain battery) and a 30 s one. The 2 m marker is rendered *area-correctly* (a sub-pixel target darkens a pixel only by the fraction it covers), so beyond ∼100 m it is invisible in the 32×24 wide view. The camera is therefore *foveated* (Fig. 4): two further 32×24 views (30° and 8°) are centred on a gaze point that the policy itself outputs.

IV. METHOD

Teacher. PPO [23] with GAE trains an MLP on the body-frame state. The critic also receives the hidden parameters, the wind and the time left [15]. The reward pays progress toward the target, a time penalty, success (+10) and crashes (−10), so the fastest safe approach maximises return. A small perception term rewards keeping the target near the optical axis. At long range a curriculum moves from short to long targets. A *visibility-limited* teacher, warm-started from the full one, knows the target only while it is inside the wide view and within 150 m, and otherwise acts on a belief (the last sighting, or a prior). It reaches 99.5% on the standard benchmark (all-knowing: 99.9%). Its visibility test is geometric rather than perceptual, so it remains partially privileged.

Students. A CNN (3 strided convolutions, shared across views) processes the frame stack and is concatenated with the student's own last H commands and current gaze. A memoryless student maps these to sticks and next gaze with an MLP; a recurrent student feeds them through a 256-unit GRU first. **DAgger** [16] rolls out the mixture policy (the teacher acts with probability $\beta \to 0$) and labels every visited state with the teacher's action. Labels are aggregated in a ring buffer: without aggregation the student collapsed when β reached zero. Recurrent students are trained on 32-step sequences stored with their initial hidden state.

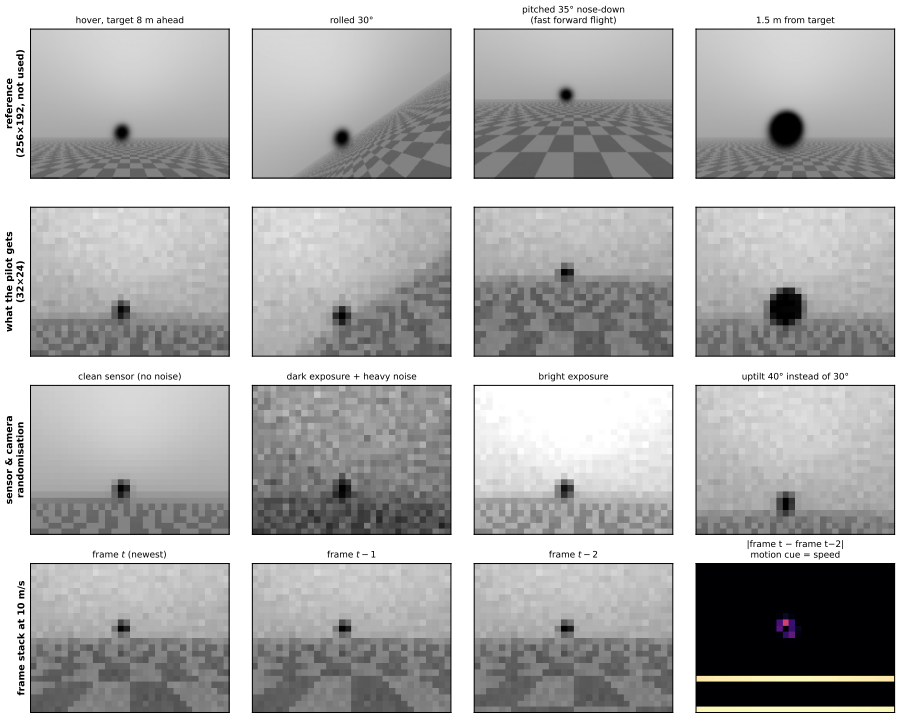


Fig. 3. What the pilot sees (short-range camera). Row 1: high-resolution reference (not used). Row 2: the 32×24 input for the same poses. Row 3: sensor randomisation. Row 4: a frame stack at 10 m/s and its temporal difference.

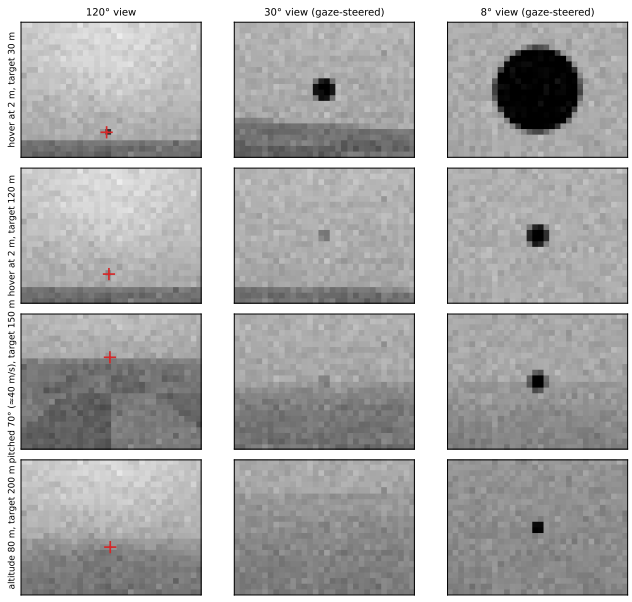


Fig. 4. Long-range camera: the 120° wide view (cross: gaze) and the gaze-steered 30° and 8° views. The 2 m marker vanishes from the wide view beyond ∼100 m but spans pixels in the 8° view.

RL fine-tuning. The DAgger student is then trained with PPO on its own observations: a Gaussian over sticks and gaze, the privileged critic warm-started from the teacher's, recurrent PPO on stored sequences, and an imitation anchor to the teacher's labels decayed from 1 to 0.1. Fine-tuning runs for 200 iterations (1.6 M steps). This lets the student optimise for what *it* can observe rather than copy a teacher it cannot fully imitate.

Algorithm 1 Training a camera-only pilot

1: train teacher π_T with PPO on the true state (asymmetric critic)
2: $\mathcal{D} \leftarrow \emptyset$; student π_S (CNN, optionally GRU)
3: **for** DAgger iteration $i = 1 \dots N$ **do** ▷ β: 1→0
4: roll out: teacher acts w.p. β, else π_S (sticks and gaze)
5: label every visited state with π_T; add to ring buffer $\mathcal{D}$
6: Adam steps on $\|\pi_S(o) - a^T\|^2$ (sequences for the GRU)
7: **end for**
8: optional: more DAgger with β=0, **or** PPO on π_S with privileged critic and a decaying imitation anchor

TABLE II
TRAINING SETTINGS (LONG RANGE)

Teacher PPO	1024 envs × 128 steps, 4 epochs × 8 minibatches, clip 0.2, γ=0.995, λ=0.95
Teacher nets	MLP 256-256 (tanh), Adam 3e-4→3e-5, 400 iterations (+200 visibility-limited)
Student nets	3 convs (16/32/32 ch., stride 2) shared over views; MLP 256-256 or GRU 256
DAgger	512 envs × 32 steps, 600 iterations × 16 Adam steps (batch 2048), buffer 12 iterations, β: 1→0 over 250 iterations, lr 3e-4 cosine
More DAgger (control)	100 iterations (1.6 M steps), β=0, lr 3e-5 cosine
RL fine-tune	256 envs × 32 steps, 200 iterations (1.6 M steps), clip 0.1, actor lr 2e-5, critic lr 3e-4, anchor weight 1→0.1
Evaluation	1024 episodes per benchmark, deterministic policy mean

V. SHORT-RANGE VALIDATION

On the short task (Table III, one training seed, 1024 episodes) the camera-only student reaches 79.8% (teacher 100.0%; 96.0% on the nominal drone). 99.0% of its failures pass within 1.5 m of the target: the gap is terminal precision at speed, not perception. Aggregated DAgger is essential (behaviour cloning: 16.4%), frame stacking helps, and the perception reward leaves the teacher unchanged but raises its student from 67.1% to 79.8%, because it makes the expert

TABLE III
SHORT-RANGE VALIDATION (1024 EPISODES, ONE TRAINING SEED)

Policy	Success %	Crash %	Time (s)
Teacher (true state)	100.0	0.0	0.85
Student (camera), DAgger	79.8	11.5	1.59
behaviour cloning	16.4	83.6	0.79
single frame	66.0	17.3	1.78
no perception reward	67.1	25.8	1.38
Teacher, no domain rand.	91.3	5.2	1.17

imitable. Out of its training range (Fig. 5) the student is robust to wind but fragile to latency (66% at the maximum training latency, 8% at twice it), and to thrust-to-weight and camera uptilt: the axes that require identifying the drone online.

VI. LONG RANGE: MEMORY AND FURTHER TRAINING

Protocol. Every run uses the same settings:

- the visibility-limited teacher, 3-view foveated camera and stick history $H{=}10$;
- DAgger for 600 iterations of 16 updates, with $\beta \to 0$ over 250 iterations;
- 1024 evaluation episodes per benchmark, identical for every policy.

We cross memory (none / GRU) with the training stage, over 5 seeds:

- imitation only;
- + RL fine-tuning;
- + a *compute-matched control* that continues DAgger (teacher no longer acting, learning rate at its final value) for the same 1.6 M steps instead.

Both follow-up stages start from the same imitation checkpoint, so their comparison is paired. As a non-learned baseline, a *visual-servoing* pilot sees the same views:

- it estimates attitude from a horizon fit (roll correlates at 0.99 with the truth) and finds the target as the darkest blob in the narrowest view;
- it flies pursuit guidance with line-of-sight damping, with gains tuned by grid search on separate episodes;
- it is *calibrated* with the true camera uptilt and thrust-to-weight, which the students never receive.

Results (Table IV, Fig. 7):

- *Memory is the robust ingredient.* Imitation success rises from $44.8 \pm 12.9\%$ to $66.5 \pm 6.8\%$ (+21.7 points, Welch t=3.3). The memory advantage persists after further training, though it shrinks: after RL it is +9.6 points (t=1.9).
- *Training on the student's own states helps.* Continuing DAgger for 1.6 M steps with the teacher no longer acting adds +15.5 points without memory and +7.7 with memory, in 5 and 5 of 5 seeds respectively. Most of the remaining imitation gap is distribution shift, not a lack of capacity.
- *RL fine-tuning adds a small extra.* With the same budget, RL instead of further DAgger adds $+7.3 \pm 7.2$ points without memory and $+3.0 \pm 4.1$ with memory, better in 4 and 4 of 5 seeds. It is consistent but modest; without the control, we would have attributed the whole +10.7 points to RL.
- *Learning matters.* The best student, memory with RL, reaches $77.2 \pm 10.2\%$ (30 s: $85.6 \pm 6.3\%$). The calibrated visual-servoing pilot reaches 3.0%, crashing in 90.0% of episodes: its horizon-based attitude estimate fails at the steep pitch angles of 40 m/s flight.
- *What remains:* search failures (17.6% timeouts: the student loses the target and circles, Fig. 6), a crash floor of about 5%, and speed (7.93 s vs. 4.94 s for the teacher).

Training dynamics. Fig. 8 shows the DAgger hand-over: while the teacher still acts, the student imitates it closely (imitation error $\approx$0.003); as β reaches zero the student meets states the teacher never visits, the error rises about $100\times$, and success dips before recovering as the aggregated buffer fills with the student's own states. This is why further training on those states (more DAgger or RL) still pays off after 600 iterations.

Failure modes. All students reach the teacher's top speed (peak $\approx$38.7 m/s), so the gap is not timidity. Most failures are *search* failures: 66.3% of the memory student's timeouts (63.7% without memory) never come within 20 m of the target. Fine-tuning reduces these most for the memoryless student (to 39.1%), consistent with RL learning to re-acquire a lost target. Crashes, typically ground strikes on low targets, stay at about 5%.

What the memory encodes. We fit ridge readouts from the GRU state to quantities no student is given, on held-out episodes, and as a control fit the same probes on the memoryless students' penultimate features (Table V). Shuffled targets give $R^2 \approx 0$. Attitude and body rates are equally decodable without memory: the current horizon and stick history determine them. What memory adds is *velocity* (R^2 $0.57 \to 0.79$), the quantity that requires integration over time and that a pursuing pilot needs to anticipate where the target will be. Neither architecture encodes the drone's parameters (thrust-to-weight, latency, wind), so the memory does not perform online system identification.

VII. DISCUSSION AND LIMITATIONS

What a camera-only pilot needs. Memory matters most: the current frames reveal attitude but not velocity, and a pursuing pilot needs velocity to anticipate where the target will be. Beyond that, the student benefits from training on the states it actually visits. Continued DAgger provides most of this benefit and RL a little more, consistent with RL optimising for what the student can observe. The compute-matched control was essential: it reduced the apparent RL gain from +10.7 to +3.0 points. **Threats to validity.**

- *Simulation only*, with a simple scene (no motion blur, rolling shutter or clutter), a first-order rate loop and a single static target. Absolute success rates should not be expected to transfer; the qualitative findings, which concern the information structure of the task, are the most likely to.
- *Statistics:* with 5 seeds, the memory effect on imitation is significant (t=3.3); the RL-over-DAgger gains are con-

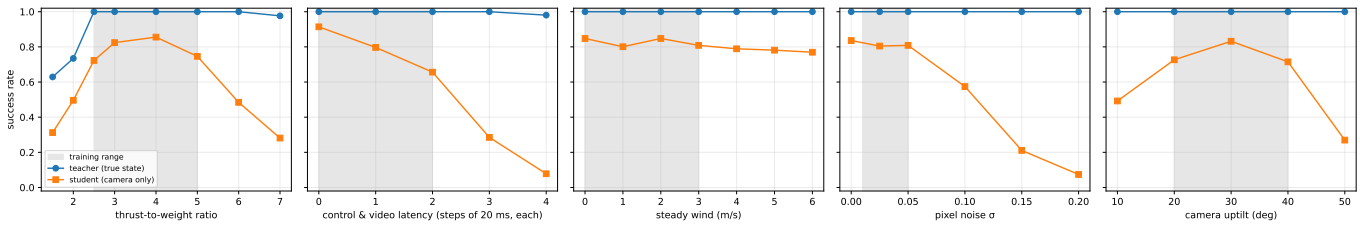


Fig. 5. Short range: success when one parameter is fixed (others randomised), 256 episodes per point, one training seed. Grey: training range.

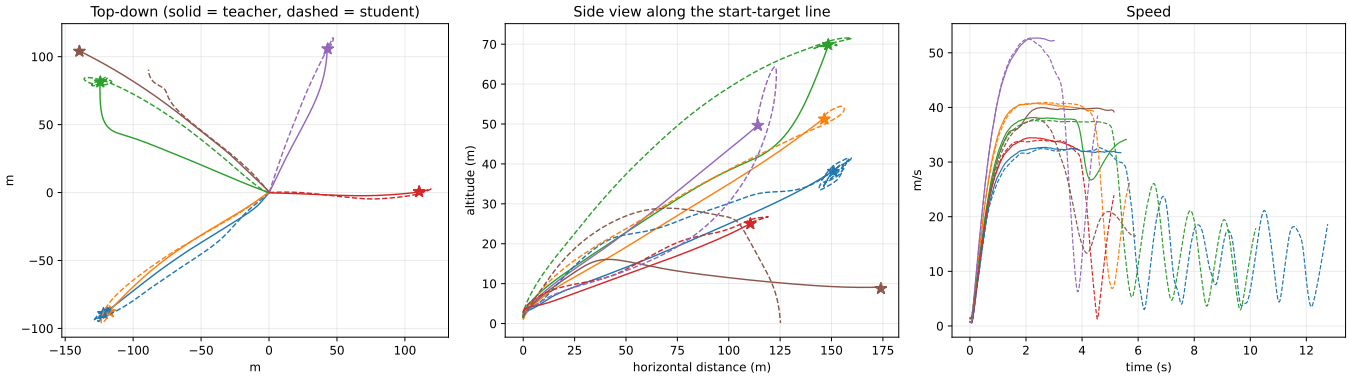


Fig. 6. Six long-range episodes. Solid: visibility-limited teacher. Dashed: camera-only student (memory + RL). Left: top-down. Middle: altitude along the start–target line (stars: targets). Right: speed. The student flies teacher-like approaches at up to 40–50 m/s; failures overshoot, lose the target and circle.

TABLE IV

LONG RANGE, CAMERA ONLY: MEAN ± S.D. OVER 5 SEEDS (1024 EPISODES PER BENCHMARK, IDENTICAL FOR ALL POLICIES). TEACHERS ON THE SAME EPISODES: VISIBILITY-LIMITED 99.5%, ALL-KNOWING 99.9% (15 s). VISUAL-SERVOING BASELINE: 3.0% (15 s), 6.1% (30 s).

		Standard (15 s)				30 s	
Memory	Training	Success %	Crash %	Timeout %	180 m+ %	Success %	Time to target (s)
No memory	imitation	44.8 ± 12.9	9.1	46.1	28.0	60.4 ± 11.6	9.23
	+ more DAgger	60.3 ± 6.9	5.6	34.1	47.6	77.7 ± 4.0	8.93
	+ RL fine-tune	67.5 ± 5.5	4.4	28.1	61.2	81.7 ± 3.2	8.56
Memory (GRU)	imitation	66.5 ± 6.8	6.4	27.1	55.1	76.6 ± 6.0	7.61
	+ more DAgger	74.2 ± 6.6	6.6	19.2	63.7	83.2 ± 3.8	7.43
	+ RL fine-tune	77.2 ± 10.2	5.2	17.6	68.7	85.6 ± 6.3	7.93

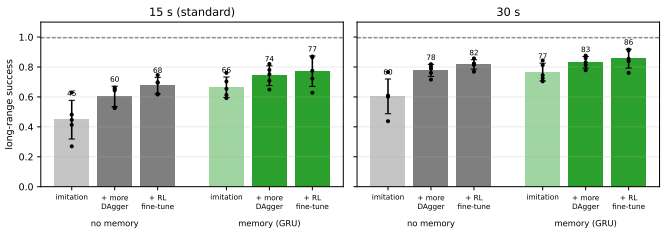


Fig. 7. Long-range success over 5 seeds (bars: mean; whiskers: s.d.; dots: seeds; dashed: teacher). "More DAgger" is the compute-matched control for RL fine-tuning.

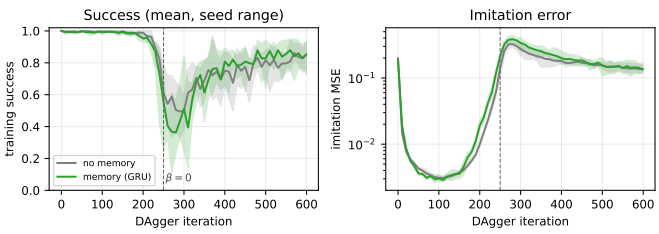


Fig. 8. Long-range DAgger training over 5 seeds (lines: mean; bands: seed range). Dashed: the teacher stops acting ($\beta = 0$).

sistent in sign (4 of 5 seeds) but small relative to the seed spread.

- *Baselines:* the short-range results are single-seed, and stronger baselines (pixel RL without a teacher, a state-estimation pipeline) remain to be added.
- *Teacher:* the visibility-limited teacher is still partially privileged.

TABLE V
LINEAR-PROBE R^2 (HELD-OUT EPISODES, MEAN $\pm$ S.D. OVER SEEDS)

Quantity	No memory	Memory	Memory + RL
attitude (gravity direction)	0.90 $\pm$ 0.01	0.86 $\pm$ 0.02	0.81 $\pm$ 0.03
body rates	0.93 $\pm$ 0.01	0.92 $\pm$ 0.01	0.91 $\pm$ 0.01
velocity (body frame)	0.57 $\pm$ 0.03	0.79 $\pm$ 0.01	0.79 $\pm$ 0.02
speed	0.50 $\pm$ 0.05	0.79 $\pm$ 0.03	0.83 $\pm$ 0.02
altitude	0.51 $\pm$ 0.07	0.52 $\pm$ 0.07	0.56 $\pm$ 0.07
distance to target (log)	0.89 $\pm$ 0.03	0.92 $\pm$ 0.02	0.93 $\pm$ 0.02
direction to target	0.25 $\pm$ 0.06	0.34 $\pm$ 0.05	0.38 $\pm$ 0.08
thrust-to-weight	-0.04 $\pm$ 0.06	-0.01 $\pm$ 0.08	0.04 $\pm$ 0.03
latency	0.02 $\pm$ 0.03	0.17 $\pm$ 0.06	0.18 $\pm$ 0.06
wind	-0.05 $\pm$ 0.02	-0.11 $\pm$ 0.05	-0.10 $\pm$ 0.04
chance (shuffled attitude)	-0.01 $\pm$ 0.00	-0.01 $\pm$ 0.00	-0.01 $\pm$ 0.00

Toward hardware. The interface maps directly onto a real FPV drone: the policy outputs acro-mode sticks that a Betaflight-class flight controller tracks, and its inputs are 32×24 crops that could be cut from an onboard 1080p feed, with the gaze selecting the narrow crops. The student is small: about 5 MFLOP per control step for all three views, the GRU and the head, i.e., 0.25 GFLOP/s at 50 Hz, well within embedded processors. The main gaps are visual realism (texture, blur, clutter), a real rate-loop response, and end-to-end latency, which the short-range sweep shows is the student's most sensitive axis.

Why not RL from pixels alone? The privileged teacher supplies a dense, low-variance training signal at every visited state, and the student needs only $\approx$10 M steps of imitation. RL is most useful at the end, refining the student on what it can observe, rather than discovering acro flight from pixels with a sparse reward.

Cost. The whole long-range study runs on a fanless laptop CPU (an imitation run takes 68–109 min, an RL fine-tune 9–13 min). XLA:CPU convolutions inside `lax.scan` slowed down by $\sim$25$\times$; an im2col reformulation avoids this.

Reproducibility. Physics, rendering, both networks and both learning algorithms are pure JAX functions, batched with `vmap` and compiled into one XLA program per training iteration. The code, all checkpoints' evaluation results and the scripts that generate every number in this letter from them are released. An extended version adds the short-range ablations in full, the exploratory runs (including an IMU-equipped student) and further figures.

VIII. CONCLUSION

With the camera as its only sensor, a learned acro pilot reaches 77.2% of 100–200 m targets at up to 40 m/s within 15 s, where a calibrated hand-designed pilot reaches 3.0%. The ingredient that matters most is memory, which adds a velocity estimate that the frames do not provide. Training on the student's own states adds further gains, and RL fine-tuning a small extra. Gates, obstacles, online identification of the drone [14] and hardware experiments are the next steps.

REFERENCES

[1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, pp. 982–987, 2023.

[2] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, "Reaching the limit in autonomous racing: Optimal control versus reinforcement learning," *Science Robotics*, vol. 8, no. 82, 2023.

[3] I. Geles, L. Bauersfeld, A. Romero, J. Xing, and D. Scaramuzza, "Demonstrating agile flight from pixels without state estimation," in *Robotics: Science and Systems (RSS)*, 2024.

[4] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Conference on Robot Learning (CoRL)*, 2019.

[5] J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter, "Learning quadrupedal locomotion over challenging terrain," *Science Robotics*, vol. 5, no. 47, 2020.

[6] E. Kaufmann, A. Loquercio, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Deep drone acrobatics," in *Robotics: Science and Systems (RSS)*, 2020.

[7] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Science Robotics*, vol. 6, no. 59, 2021.

[8] Y. Song, K. Shi, R. Penicka, and D. Scaramuzza, "Learning perception-aware agile flight in cluttered environments," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[9] Y. Song, S. Naji, E. Kaufmann, A. Loquercio, and D. Scaramuzza, "Flightmare: A flexible quadrotor simulator," in *Conference on Robot Learning (CoRL)*, 2020.

[10] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, "Control of a quadrotor with reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 2, no. 4, pp. 2096–2103, 2017.

[11] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement learning for UAV attitude control," *ACM Transactions on Cyber-Physical Systems*, vol. 3, no. 2, 2019.

[12] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, "Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019.

[13] J. Eschmann, D. Albani, and G. Loianno, "Learning to fly in seconds," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6336–6343, 2024.

[14] A. Kumar, Z. Fu, D. Pathak, and J. Malik, "RMA: Rapid motor adaptation for legged robots," in *Robotics: Science and Systems (RSS)*, 2021.

[15] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric actor critic for image-based robot learning," in *Robotics: Science and Systems (RSS)*, 2018.

[16] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.

[17] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[18] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[19] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac gym: High performance GPU-based physics simulation for robot learning," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[20] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on Robot Learning (CoRL)*, 2021.

[21] C. D. Freeman, E. Frey, A. Raichuk, S. Girgin, I. Mordatch, and O. Bachem, "Brax – a differentiable physics engine for large scale rigid body simulation," in *NeurIPS Datasets and Benchmarks Track*, 2021.

[22] C. Lu, J. G. Kuba, A. Letcher, L. Metz, C. Schroeder de Witt, and J. Foerster, "Discovered policy optimisation," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.